ZentroMart VoiceOps Capstone Submission Report

Project Summary

This project implements a comprehensive, voice-enabled customer support agent for ZentroMart across the four mandated capstone stages. The primary objective of this system is to automate repetitive tier-one customer service inquiries—such as order status checks, return eligibility, refund timelines, and policy clarifications—while maintaining strict safety guardrails around private customer data and discretionary compensation.

The implementation progresses through four distinct technical phases. The first phase establishes a baseline using a prompt-only Large Language Model approach, highlighting the hallucinations that occur when an agent lacks grounding. The second phase introduces Retrieval-Augmented Generation, grounding the agent in current ZentroMart policy documents and product catalog records. The third phase elevates the system to handle multi-turn conversations, enabling context retention, follow-up questions, identity-verified order lookups, and support ticket creation. Finally, the fourth phase integrates Gnani speech-to-text and text-to-speech capabilities, allowing the agent to process batches of customer audio files and respond with synthesized voice audio.

The system was built using the supplied synthetic ZentroMart dataset. A core design principle of this implementation is the strict preservation of data types; order IDs, SKUs, and pincodes are strictly handled as strings to prevent data loss. Furthermore, the retrieval engine enforces a strict hierarchy of authority, prioritizing current policy documents over older guides, frequently asked questions, marketing pages, and historical support logs.

System Architecture

The implementation is organized as a command-line Python application, engineered for modularity and auditability. The runtime execution path begins with configuration and dataset validation, proceeds to the chosen evaluation or voice-batch command, passes through the retrieval engine and conversation state controller, and finally leverages the Groq API for response generation before writing structured output artifacts.

The system is divided into several specialized modules. The configuration module loads environment variables, resolves dataset paths relative to the project root, and validates the presence of required API keys. The dataset module is responsible for loading CSV files losslessly. The knowledge base module parses Markdown documents, splitting them by section headings, and transforms tabular product catalog rows into structured evidence chunks. The retrieval module builds the semantic index and applies custom scoring logic to boost exact catalog matches. The generative module applies an evidence gate, constructs grounded prompts, and interfaces with the LLM. The conversation module maintains multi-turn state, strictly gates tool usage, and protects sensitive order information until caller identity is verified. The voice module manages the integration with Gnani APIs, handles audio format conversions, and orchestrates batch processing.

A major architectural decision was to completely separate the retrieval process from the response generation process. This decoupling ensures that every response can be independently audited against the exact section IDs, catalog rows, source files, similarity scores, and source text that were supplied to the model, preventing opaque reasoning.

Configuration and Secrets Management

Security and configuration management are handled strictly through environment variables to ensure no sensitive credentials are hardcoded into the repository. Credentials are supplied through a local environment file, which is explicitly excluded from version control. The supported configuration requires a primary Groq key, an optional fallback Groq key, the designated Groq model name, a Gnani API key for speech processing, and the absolute or relative path to the extracted ZentroMart dataset.

To ensure high availability during batch processing and evaluation runs, the LLM client implements robust rate-limit handling. When a Groq request encounters a rate limit, the client parses the provider's retry hint, applies a bounded backoff delay, and attempts the request again. If the primary key exhausts its retry allowance, the system automatically fails over to the secondary configured key. Non-rate-limit failures are allowed to raise exceptions naturally, ensuring that system errors are never silently masked as successful responses.

Dataset Handling and Knowledge Ingestion

The supplied synthetic dataset is treated as the immutable source of truth. The data loader reads all CSV fields as strings to preserve critical formatting, such as leading zeroes in pincodes and order identifiers. It separates practice evaluation data, unstructured Markdown knowledge documents, structured catalog records, and offline tool data into distinct processing streams.

The knowledge ingestion engine reads Markdown metadata fields to determine document type, status, and effective dates. A critical safety feature is the automatic exclusion of superseded, obsolete, or retired documents; they are never indexed as authoritative retrieval evidence. The system also enforces a strict policy ordering to resolve conflicting information. Current policy documents are granted the highest priority, followed by agent guides, help documents, FAQs, and finally marketing material. Historical support logs are explicitly excluded from the policy retrieval index to prevent the agent from mimicking past agent mistakes or one-off exceptions.

Evidence Model and Retrieval Method

Markdown sections are mapped to unified knowledge objects. To allow the agent to answer product-specific inquiries, each row in the product catalog is also converted into this exact same knowledge object format. A product chunk includes its SKU, title, catalog category, current status, effective date, and a fully normalized text representation of its attributes. This unified model allows a question about a specific SKU to retrieve the product's catalog row to establish its return category or warranty status, while simultaneously retrieving the overarching return or warranty policy section that dictates how that category is handled.

The retrieval engine utilizes dense semantic embeddings via SentenceTransformers to understand the intent behind customer queries, moving beyond simple keyword matching. However, to accommodate the highly specific nature of retail identifiers, the retrieval engine applies custom heuristic boosting. Exact SKU matches and exact product-title matches receive a massive artificial score boost to ensure they are always included in the context window. Partial product-title overlap is also calculated to help identify products when customers describe them without providing a SKU.

To prevent product catalog rows from crowding out the actual policy rules, the retrieval engine reserves result slots specifically for policy or help evidence. Furthermore, if a catalog row is retrieved, the engine extracts its category terms and applies a secondary boost to any policy sections that mention those specific categories.

Grounding and Uncertainty Behavior

To strictly prevent hallucinations, the system implements a conservative lexical evidence gate before any call to the LLM is made. This gate analyzes the meaningful terms in the customer's query and verifies that a sufficient percentage of those terms exist within the retrieved evidence text. If the evidence is deemed insufficient, the system short-circuits the generation process and returns a predefined uncertainty response, explicitly refusing to guess or invent an answer.

When sufficient evidence is found, the generated prompt applies heavy constraints on the model. The prompt explicitly forbids inventing timelines, refund amounts, eligibility criteria, or system capabilities. It demands that the model combine catalog facts with the policy that interprets them, and it requires the model to append source section identifiers to the end of the response for auditability. This defense-in-depth approach relies on retrieval filtering, lexical gating, prompt engineering, and output recording to guarantee grounding.

Conversation State and Tool Safety

The conversation controller is responsible for maintaining context across multiple turns while strictly gating access to the supplied mock tools. The controller stores recent messages, the currently pending order ID, the verified order details, and the count of verification failures. Order identifiers are normalized using string manipulation to remove spaces and punctuation without triggering numeric conversion errors.

A critical security mandate implemented in the controller is the protection of private order information. An initial order lookup is permitted to check if an order exists, but the controller intercepts the result and hides the private details from the LLM until the customer provides a matching verification value, such as a delivery pincode or the last four digits of their registered phone number. If verification fails repeatedly, the controller permanently blocks disclosure for that session and directs the customer to human support.

While the LLM may propose an action, the Python controller is solely responsible for tool execution. This prevents the LLM from hallucinating a successful tool call. Ticket creation is only communicated to the user as successful if the Python execution of the dataset tool explicitly returns a success flag. Any failed or ambiguous tool results force the agent to produce an unconfirmed handover response, maintaining absolute honesty with the customer.

Voice Batch Processing Pipeline

The voice integration pipeline is designed to process batches of customer audio files autonomously. The runner scans the designated input directory for supported audio formats. It initializes the Gnani speech-to-text and text-to-speech clients once per batch to optimize network overhead, but it creates a completely fresh conversation controller for each audio file to prevent data leakage between unrelated callers.

The pipeline strictly enforces the maximum audio duration limit to prevent excessive API costs. Because the Gnani speech-to-text API requires specific audio formats, the pipeline incorporates an automatic FFmpeg conversion step. Any input video or audio file that is not in the required format is extracted and down-sampled to a temporary mono audio file before transmission. Once the transcript is received, the RAG and conversation controllers generate a response, which is then sent to the Gnani text-to-speech API using the designated Indian English voice model. A heuristic fallback dynamically switches the language parameter if Hinglish vocabulary is detected in the transcript.

The output contract for the batch processor is highly structured. For every input file, it generates a raw audio reply, a plain text response, a plain text transcript, a structured JSON file containing the retrieved evidence, and a comprehensive result JSON logging the execution status, tool calls, and decisions. A final batch manifest aggregates the success and failure counts for the entire run.

Evaluation Results and Metric Analysis

The system was evaluated against the provided practice dataset to measure improvements between the prompt-only baseline and the RAG-enabled implementation.

The baseline evaluation processed all practice questions using only the system prompt, without access to the knowledge base or catalog. While the system successfully generated responses without encountering runtime errors, the semantic accuracy was inherently flawed, as the model was forced to invent ZentroMart-specific policies, resulting in high confidence but factually incorrect assertions.

The Retrieval-Augmented Generation evaluation demonstrated massive improvements in factual grounding. The system processed the answerable questions with a high grounding rate, meaning it successfully found and utilized evidence for those queries. The must-not-include pass rate reached absolute perfection, confirming that the agent successfully avoided prohibited terms and refused to invent discretionary compensation. The must-include pass rate demonstrated that the agent successfully incorporated mandatory policy terms in the majority of its answers.

The conversation evaluation tested the multi-turn state machine and tool calling capabilities. The system exhibited perfect runtime stability, successfully handling every conversation turn without throwing unhandled exceptions. The tool-call agreement rate was strong, indicating that the agent correctly decided when to look up an order and when to escalate to a ticket. Decision mismatches primarily occurred due to the agent proactively offering to create a ticket for complex returns, whereas the dataset label expected a strict policy answer first. Overall, the RAG and conversation evaluations prove that grounding the LLM entirely eliminates the hallucinations observed in the baseline step.

Limitations and Future Improvements

While the agent meets the core requirements of the capstone challenge, several limitations exist. The decision agreement metrics in the conversation evaluation indicate that behavioral alignment with the strict dataset labels is not yet perfect. The model occasionally struggles to extract heavily spaced or highly fragmented order identifiers from Hindi phrasing, leading to missed tool calls. Furthermore, the unanswerable correct-uncertainty rate highlights a strictness mismatch between the lexical evidence gate and the evaluator's expected format for out-of-domain refusals.

Future engineering efforts should focus on moving the deterministic extraction of order identifiers and verification values ahead of the LLM decision step. Using a lightweight, dedicated extraction model or advanced regular expressions for Hinglish phrases would drastically improve tool-call reliability. Additionally, expanding the retrieval logic to perform multi-hop reasoning—specifically for queries that require joining a product category, a specific order date, and a holiday calendar—would improve the gold-evidence recall metrics.

For production deployment, the voice pipeline would need to transition from batch file processing to real-time streaming. Implementing WebSocket connections for the Gnani speech-to-text and text-to-speech services would reduce latency, allowing the agent to interrupt itself and respond to the customer with natural conversational timing.